Index Prediction & Analysis of S&P 500 Using Machine Learning

Shashank Raj
Department of Computer Science and Engineering,
Michigan State University, East Lansing, MI, USA

***Abstract*—This paper investigates the application of Long Short-Term Memory (LSTM) neural networks to financial index prediction and algorithmic trading. A complete system is developed that includes data preprocessing, LSTM model training, trading strategy formulation, and backtesting. The system is applied to the S&P 500 index, demonstrating how deep learning can capture complex market dynamics and generate actionable trading signals. The performance of the proposed strategy is carefully evaluated, and the theoretical limits of predictability are explored using an optimal trade sequence algorithm.**

***Index Terms*—S&P 500, Machine Learning, LSTM, Time Series Analysis, Financial Prediction, Algorithmic Trading, Backtesting, Optimal Trade Sequence.**

I. Introduction

Predicting the behavior of financial markets is both challenging and important, as it impacts investment strategies, risk management, and economic decision-making. Financial time series, however, come with unique characteristics that make forecasting a tough problem to crack. In this paper, we introduce a deep learning framework aimed at addressing these challenges and enhancing the prediction of financial indices.

Financial markets are complex systems influenced by a wide range of factors such as economic indicators, investor behavior, global events, and market-specific dynamics. These factors generate time series data that behave very differently from many other types of data. Some of the main features include:

- **Non-stationarity:** The statistical properties of financial data, like the mean and variance, change over time (1). This variability makes it difficult to use traditional models that rely on constant behavior.
- **High Noise-to-Signal Ratio:** Financial data is full of random fluctuations that can obscure true trends, making it hard to extract meaningful patterns.
- **Volatility Clustering:** Calm periods in the market are often followed by calm periods, and turbulent periods are likely to continue (2). This inconsistent behavior challenges many common forecasting methods.
- **Fat Tails:** Extreme price movements occur more frequently than standard models would predict. Accounting for these rare but significant events is crucial for robust forecasting and risk management.
- **Limited Predictability:** According to the Efficient Market Hypothesis, market prices quickly reflect all available information, which makes consistent prediction very challenging. Although markets may not be perfectly efficient, they are still hard to predict.

Shashank Raj is with the Department of Computer Science and Engineering, Michigan State University, East Lansing, MI, USA.

Traditional methods like ARIMA and GARCH often fall short because they assume stable relationships that rarely hold true in real financial markets. In contrast, deep learning models, particularly LSTM networks, excel at handling non-linear patterns and long-term dependencies, making them a promising tool for financial forecasting.

Our research uses LSTM networks to predict movements in the S&P 500 index and to develop a trading strategy based on these predictions. Given the S&P 500's role as a key benchmark for the U.S. stock market, accurate predictions can offer significant benefits to investors.

This paper covers the full development cycle of a deep learning-based trading system, including data collection and preprocessing, model training, strategy backtesting, and performance evaluation. A significant contribution of this work is the creation of a flexible, Python-based framework that integrates these components seamlessly.

The remainder of the paper is organized as follows: Section II introduces the theoretical framework behind financial time series and LSTM networks; Section III discusses the system implementation; Sections IV through VI detail the data processing, model architecture, training process, and evaluation metrics; Sections VII and VIII present the S&P 500 case study and trading strategy; Sections IX and X analyze the results and discuss limitations; finally, Section XI outlines future improvements and Section XII concludes the paper.

II. Theoretical Framework

A. Time Series Properties in Financial Markets

Stock market prediction is fundamentally a time series forecasting problem. Unlike traditional time series that may exhibit clear seasonal patterns, financial time series are marked by several unique characteristics:

- **Non-stationarity:** The statistical properties (mean, variance, etc.) change over time. This requires special techniques to handle shifting data distributions (1).

- **High Noise-to-Signal Ratio:** Market data is frequently overwhelmed by random fluctuations, making it difficult to distinguish meaningful signals.
- **Regime Shifts:** Abrupt changes in market behavior occur during transitions (e.g., bull to bear markets), which can drastically alter the underlying dynamics.
- **Fat-Tailed Distributions:** Extreme events (large price moves) occur more frequently than would be predicted by a normal distribution (3).
- **Heteroscedasticity:** The variance of returns is not constant over time; high volatility periods often cluster together.

B. Mathematical Representation of Financial Time Series

The price process of a financial asset can be modeled as:

$$P_t = P_{t-1} + \mu_t + \epsilon_t, \tag{1}$$

where:

- P_t is the price at time t.
- μ_t is the drift term representing the expected return.
- ϵ_t is the random error term (noise).

For predictive modeling, we often analyze returns rather than raw prices. The simple return is defined as:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}} = \frac{P_t}{P_{t-1}} - 1. \tag{2}$$

Log returns, which tend to offer better statistical properties, are given by:

$$R_t = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln(P_t) - \ln(P_{t-1}). \tag{3}$$

Log returns are approximately additive over time and are closer to a normal distribution, making them more suitable for statistical analysis (4).

C. Autocorrelation and Predictability

Autocorrelation measures the similarity between observations as a function of the time lag between them. The autocorrelation function (ACF) for a lag k is defined as:

$$\rho(k) = \frac{E\left[(r_t - \mu)(r_{t-k} - \mu)\right]}{\sigma^2}, \tag{4}$$

where:

- μ is the mean return.
- σ^2 is the variance.

Significant autocorrelation at specific lags suggests that past information may help in predicting future returns (5). However, in efficient markets, such autocorrelation is usually weak and diminishes rapidly with increasing lag.

D. Volatility Clustering and GARCH Models

Volatility clustering refers to the tendency of high-volatility events to group together (2). This behavior is often modeled using GARCH (Generalized Autoregressive Conditional Heteroskedasticity) models (6). A simple GARCH(1,1) model can be expressed as:

$$\sigma_t^2 = \omega + \alpha\epsilon_{t-1}^2 + \beta\sigma_{t-1}^2, \tag{5}$$

where:

- σ_t^2 is the conditional variance at time t.
- ω is a constant term.
- α measures the impact of new information (the effect of previous errors).
- β captures the persistence of past volatility.

Understanding and modeling these volatility dynamics is crucial for both risk management and derivative pricing (7). Our LSTM model, while not explicitly modeling volatility like GARCH, learns these complex temporal patterns implicitly.

E. LSTM Networks: Mathematical Foundations

LSTM (Long Short-Term Memory) networks are a type of Recurrent Neural Network (RNN) designed to overcome the limitations of standard RNNs, particularly the vanishing gradient problem.

1) RNN Limitations and the Vanishing Gradient Problem: While RNNs are intended to process sequences of arbitrary length, they often struggle with long-term dependencies due to the vanishing gradient problem (8). For a recurrent weight matrix W, the gradient at time step t is:

$$\frac{\partial L}{\partial W} = \sum_{k=1}^{t} \frac{\partial L}{\partial y_t}\frac{\partial y_t}{\partial h_t}\frac{\partial h_t}{\partial h_k}\frac{\partial h_k}{\partial W}, \tag{6}$$

with:

$$\frac{\partial h_t}{\partial h_k} = \prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}} = \prod_{j=k+1}^{t} W^T \operatorname{diag}\left(f'(h_{j-1})\right). \tag{7}$$

If the largest eigenvalue of W is less than 1, the product tends to zero as the time difference $t-k$ increases, leading to vanishing gradients. Conversely, if it is greater than 1, gradients may explode.

2) LSTM Architecture: LSTMs address the vanishing gradient problem using a memory cell and gating mechanisms that regulate the flow of information (9). The LSTM cell comprises several components:

a) Input Gate::

$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] + b_i\right) \tag{8}$$

b) Forget Gate::

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right) \tag{9}$$

c) Candidate Cell State::

$$\tilde{C}_t = \tanh\left(W_C \cdot [h_{t-1}, x_t] + b_C\right) \tag{10}$$

d) Cell State Update::

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \tag{11}$$

where $\odot$ represents element-wise multiplication (10).

e) Output Gate and Hidden State Update::

$$o_t = \sigma\left(W_o \cdot [h_{t-1}, x_t] + b_o\right) \tag{12}$$

$$h_t = o_t \odot \tanh(C_t). \tag{13}$$

In our implementation, we use a stacked LSTM architecture with two layers:

$$h_t^{(1)} = \text{LSTM}_1\left(x_t, h_{t-1}^{(1)}, C_{t-1}^{(1)}\right), \tag{14}$$

$$h_t^{(2)} = \text{LSTM}_2\left(h_t^{(1)}, h_{t-1}^{(2)}, C_{t-1}^{(2)}\right), \tag{15}$$

$$\hat{y}_t = W_o\, h_t^{(2)} + b_o, \tag{16}$$

where $\hat{y}_t$ is the predicted output.

3) Backpropagation Through Time (BPTT): Training LSTM networks requires an extension of standard backpropagation called Backpropagation Through Time (BPTT). In BPTT, the gradients of the loss function with respect to the LSTM parameters θ are computed as:

$$\frac{\partial L}{\partial \theta} = \sum_{t=1}^{T} \frac{\partial L}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial \theta}, \tag{17}$$

where T is the total number of time steps. BPTT allows the gradients to flow backwards through the sequence, enabling the network to learn from long-term dependencies.

4) Loss Function for Price Prediction: For our price prediction task, we use the Mean Squared Error (MSE) as the loss function:

$$L = \frac{1}{T} \sum_{t=1}^{T} \left(y_t - \hat{y}_t\right)^2, \tag{18}$$

where y_t is the actual price at time t, and $\hat{y}_t$ is the predicted price.

This comprehensive theoretical framework lays the foundation for our deep learning approach, merging insights from time series analysis, volatility modeling, and advanced LSTM architectures to address the complexities of financial market prediction.

F. Efficient Market Hypothesis and Statistical Arbitrage

When developing this project, we took into account the Efficient Market Hypothesis (EMH), which proposes that stock prices already incorporate all available information. In other words, if the market were perfectly efficient, it would be nearly impossible to consistently achieve returns that beat the overall market. The EMH is generally categorized into three distinct forms (11):

- **Weak form:** This form asserts that past price information (such as historical stock prices or returns) is fully reflected in current prices. Therefore, analyzing past data alone cannot provide an edge for predicting future prices.
- **Semi-strong form:** According to this form, all publicly available information is already accounted for in stock prices. This includes not just past prices, but also financial statements, news, and other public disclosures.
- **Strong form:** This most extreme version states that all information—public as well as private—is reflected in the current price. In such a market, even insiders with access to confidential information would not be able to consistently earn abnormal returns.

a) Mathematical Formalization of EMH: Under the assumptions of the EMH, stock prices are modeled as a *martingale process* (12). This means that the best prediction for tomorrow's price is simply today's price, adjusted only by random noise. Mathematically, this is expressed as:

$$E\left[P_{t+1} \mid \mathcal{F}_t\right] = P_t, \tag{19}$$

where P_{t+1} is the price at time $t+1$, P_t is the price at time t, and $\mathcal{F}_t$ represents all the information available at time t.

This formulation implies that the expected return based on past information is zero:

$$E\left[r_{t+1} \mid \mathcal{F}_t\right] = 0, \tag{20}$$

with r_{t+1} representing the return from time t to $t+1$. Essentially, if the market were perfectly efficient, there would be no opportunity to generate excess returns by simply analyzing past information.

In practical terms, this means that the best forecast for tomorrow's price would be:

$$P_{t+1} = P_t + \epsilon_{t+1}, \tag{21}$$

where ϵ_{t+1} is a random error term capturing unpredictable factors.

b) Market Inefficiencies and Statistical Arbitrage: While the EMH provides a solid theoretical foundation, real-world markets are often not perfectly efficient—especially in the short term. Temporary inefficiencies can arise due to several factors (12):

- **Behavioral biases:** Investors may overreact or underreact to new information, causing prices to deviate from their "true" value.
- **Liquidity constraints:** Large institutional investors might be forced to trade in ways that are not entirely price-optimal due to liquidity needs.
- **Information asymmetry:** Not all market participants have access to the same quality or speed of information.
- **Technical factors:** Market microstructure issues such as order imbalances or specific chart patterns can create temporary price pressures.

These inefficiencies open the door for *statistical arbitrage* strategies. Statistical arbitrage involves identifying and exploiting statistical patterns that deviate from the random walk hypothesis (13). In a nutshell, a successful arbitrage strategy aims to capture an "alpha," or excess return (14), defined as:

$$\alpha = R_p - \left(R_f + \beta(R_m - R_f)\right), \tag{22}$$

where:

- R_p is the return of the portfolio.

- R_f is the risk-free rate.
- R_m is the market return.
- β is the sensitivity of the portfolio to market movements.

A positive value of α indicates that the strategy outperforms what would be expected given the inherent risk, thereby suggesting that the temporary market inefficiencies have been successfully exploited.

In summary, while the EMH suggests that consistent abnormal returns should not be possible, real-world market imperfections create opportunities for statistical arbitrage. Our project leverages these opportunities by developing algorithmic strategies that attempt to capture such deviations, providing a pathway to potentially profitable trading despite the challenges posed by efficient markets.

G. Technical Indicators: Mathematical Basis

Technical indicators are essential tools in financial analysis, as they provide additional insights beyond raw price data. They help us to identify trends, measure momentum, gauge volatility, and spot potential reversal points. In this section, we describe the mathematical basis behind several widely used technical indicators.

1) Moving Averages: Moving averages smooth out price data to highlight trends by filtering out the noise from short-term fluctuations.

a) Simple Moving Average (SMA):: The SMA over a period of n is calculated by averaging the prices over the past n time steps (15):

$$\mathrm{SMA}_t(n) = \frac{1}{n}\sum_{i=0}^{n-1} P_{t-i}, \tag{23}$$

where P_{t-i} is the price at time $t-i$. This simple average provides a baseline to determine the overall direction of the market.

b) Exponential Moving Average (EMA):: Unlike the SMA, the EMA gives more weight to recent prices, making it more responsive to new information. It is defined recursively as:

$$\mathrm{EMA}_t(n) = \alpha \cdot P_t + (1-\alpha)\cdot \mathrm{EMA}_{t-1}(n), \tag{24}$$

where the smoothing factor α is calculated by:

$$\alpha = \frac{2}{n+1}. \tag{25}$$

This weighting scheme ensures that the most recent prices have a larger impact on the EMA.

2) Relative Strength Index (RSI): The Relative Strength Index (RSI) is a momentum oscillator that measures the magnitude of recent price changes to determine overbought or oversold conditions. It is calculated as:

$$\mathrm{RSI} = 100 - \frac{100}{1+RS}, \tag{26}$$

where RS (Relative Strength) is the ratio of the average gain to the average loss over a specified number of periods, typically 14:

$$RS = \frac{\text{Average Gain}}{\text{Average Loss}}. \tag{27}$$

For the initial calculation:

$$\text{Average Gain} = \frac{1}{n}\sum_{i=1}^{n} \text{Gain}_i, \tag{28}$$

$$\text{Average Loss} = \frac{1}{n}\sum_{i=1}^{n} \text{Loss}_i. \tag{29}$$

For subsequent calculations, a smoothing approach is used:

$$\text{Average Gain}_t = \frac{(\text{Previous Average Gain} \times (n-1)) + \text{Current Gain}}{n}, \tag{30}$$

$$\text{Average Loss}_t = \frac{(\text{Previous Average Loss} \times (n-1)) + \text{Current Loss}}{n}. \tag{31}$$

The RSI oscillates between 0 and 100, with high values typically indicating overbought conditions and low values suggesting oversold conditions.

3) Moving Average Convergence Divergence (MACD): The MACD is a trend-following momentum indicator that shows the relationship between two EMAs. It is composed of three parts:

- **MACD Line:** The difference between two EMAs of different periods (16):

$$\text{MACD Line} = \mathrm{EMA}_{12}(P) - \mathrm{EMA}_{26}(P). \tag{32}$$

- **Signal Line:** An EMA (typically 9 periods) of the MACD Line:

$$\text{Signal Line} = \mathrm{EMA}_9(\text{MACD Line}). \tag{33}$$

- **MACD Histogram:** The difference between the MACD Line and the Signal Line:

$$\text{MACD Histogram} = \text{MACD Line} - \text{Signal Line}. \tag{34}$$

The MACD helps identify potential buy or sell signals when the MACD Line crosses the Signal Line.

4) Bollinger Bands: Bollinger Bands provide a visual representation of volatility by placing bands above and below a moving average. They consist of:

- **Middle Band:** This is typically the SMA of the price over n periods:

$$\text{Middle Band} = \mathrm{SMA}_n(P). \tag{35}$$

- **Upper Band:** Calculated by adding a multiple of the standard deviation σ_n to the middle band:

$$\text{Upper Band} = \text{Middle Band} + (k \times \sigma_n), \tag{36}$$

- **Lower Band:** Calculated by subtracting a multiple of the standard deviation from the middle band:

$$\text{Lower Band} = \text{Middle Band} - (k \times \sigma_n), \tag{37}$$

where σ_n is the standard deviation of the price over n periods and k is usually set to 2 (17). These bands widen during periods of high volatility and narrow during calmer periods, providing insight into the current market state.

5) Average True Range (ATR): The Average True Range (ATR) measures market volatility by considering the full range of price movement. It begins by calculating the True Range (TR) for each period:

$$\mathrm{TR}_t = \max\left[(H_t - L_t),\ |H_t - C_{t-1}|\,,\ |L_t - C_{t-1}|\right], \quad (38)$$

where:

- H_t is the high price at time t.
- L_t is the low price at time t.
- C_{t-1} is the previous period's closing price.

The ATR is then computed as an exponential moving average of the TR (18):

$$\mathrm{ATR}_t = \frac{(n-1) \times \mathrm{ATR}_{t-1} + \mathrm{TR}_t}{n}, \quad (39)$$

which smooths the TR values over a period of n to reflect ongoing volatility levels.

a) Summary:: By incorporating these technical indicators—moving averages (both SMA and EMA), RSI, MACD, Bollinger Bands, and ATR—into our feature set, we provide our model with a rich set of information. This not only captures the trend and momentum but also the volatility and potential reversal points of the market. Such comprehensive features are critical for enhancing the model's ability to understand and predict complex market dynamics.

III. Implementation

This section details the overall system implementation, covering the complete data processing and modeling pipeline—from raw data acquisition to final strategy evaluation and visualization.

Below is an overview of the end-to-end pipeline (see Figure 1). Each component of the pipeline is designed to be modular, making it easy to update or extend individual parts without affecting the entire system.

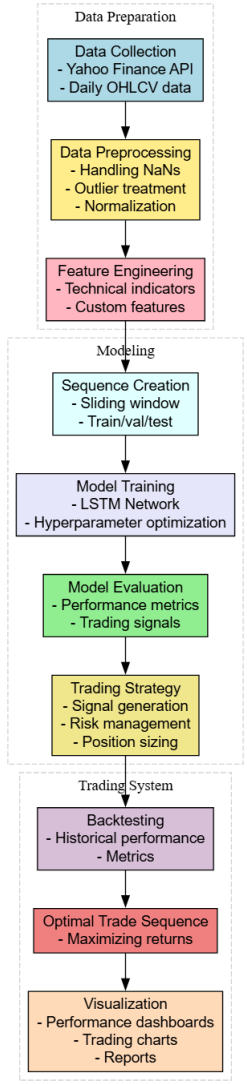

Fig. 1. Overview of the complete data processing and modeling pipeline.

A. Data Processing

Our system begins with data acquisition. The primary source is Yahoo Finance, accessed through the `yfinance` API (19). This API provides:

- Historical Price Data (Open, High, Low, Close)
- Trading Volume
- Dividend and Split Adjustments
- Basic Company Metrics

We collected approximately 2 years of S&P 500 data (2023–2025) to ensure good training samples while remaining relevant to current market conditions.

B. Technical Indicators

Technical indicators are calculated to enrich raw price data with additional market insights (20). The following pseudo-algorithms describe the computation of several key indicators.

1) Relative Strength Index (RSI): The RSI measures the magnitude of recent price changes to indicate overbought or oversold conditions (21).

Algorithm 1 Calculate RSI

procedure CalculateRSI(price_series, window)
  $\Delta \leftarrow$ Difference between consecutive prices
  $Gain \leftarrow$ Set negative values in Δ to 0
  $Loss \leftarrow$ Set positive values in Δ to 0 (take absolute value)
  $AvgGain \leftarrow$ Rolling mean of $Gain$ over $window$
  $AvgLoss \leftarrow$ Rolling mean of $Loss$ over $window$
  $RS \leftarrow AvgGain / AvgLoss$
  $RSI \leftarrow 100 - \dfrac{100}{1 + RS}$
  return RSI
end procedure

2) Moving Average Convergence Divergence (MACD): MACD is a momentum indicator that shows the relation-

ship between two EMAs.

Algorithm 2 Calculate MACD

procedure CALCULATEMACD(price_series, fast=12, slow=26, signal=9)
    $EMA_{fast} \leftarrow$ EMA of *price_series* with span *fast*
    $EMA_{slow} \leftarrow$ EMA of *price_series* with span *slow*
    $MACD\_Line \leftarrow EMA_{fast} - EMA_{slow}$
    Signal_Line $\leftarrow$ EMA of $MACD\_Line$ with span *signal*
    $Histogram \leftarrow MACD\_Line - Signal\_Line$
    return DataFrame {MACD_Line, Signal_Line, Histogram}
end procedure

3) Bollinger Bands: Bollinger Bands help identify volatility by computing a moving average and standard deviations around it.

Algorithm 3 Calculate Bollinger Bands

procedure CALCULATEBOLLINGERBANDS(price_series, window=20, num_std=2)
    MiddleBand $\leftarrow$ Rolling mean (SMA) of *price_series* ove
    StdDev $\leftarrow$ Rolling standard deviation of *price_series* ov
    $UpperBand \leftarrow MiddleBand + (num\_std \times StdDev)$
    $LowerBand \leftarrow MiddleBand - (num\_std \times StdDev)$
    return DataFrame {MiddleBand, UpperBand, LowerBand}
end procedure

4) Average True Range (ATR): ATR measures market volatility by calculating the True Range (TR) and then smoothing it using an exponential moving average.

Algorithm 4 Calculate ATR

procedure CALCULATEATR(high, low, close, window=14)
    $TR1 \leftarrow high - low$
    $TR2 \leftarrow |high - \text{shift}(close, 1)|$
    $TR3 \leftarrow |low - \text{shift}(close, 1)|$
    $TR \leftarrow \max(TR1, TR2, TR3)$
    $ATR \leftarrow$ EMA of TR over *window*
    return ATR
end procedure

C. Modeling and Strategy Evaluation

After processing the data and engineering features with the above indicators, the enriched dataset is fed into our predictive model—an LSTM network designed to capture complex market dynamics (22). The model's predictions are then used to inform an algorithmic trading strategy. The performance of this strategy is evaluated through rigorous backtesting, incorporating realistic assumptions such as transaction costs and market impact (23).

Overall, the implementation framework described above establishes a robust pipeline that converts raw financial data into actionable insights. By combining meticulous data processing, advanced technical indicators, and deep learning-based modeling, our system aims to provide a competitive edge in predicting market movements and executing profitable trades.

IV. Data Processing

This section outlines the data pipeline used in our financial index prediction system, covering everything from data acquisition to final sequence preparation for LSTM models. Figures 2, 3, and 4 show visual representations of the overall workflow and feature scaling.

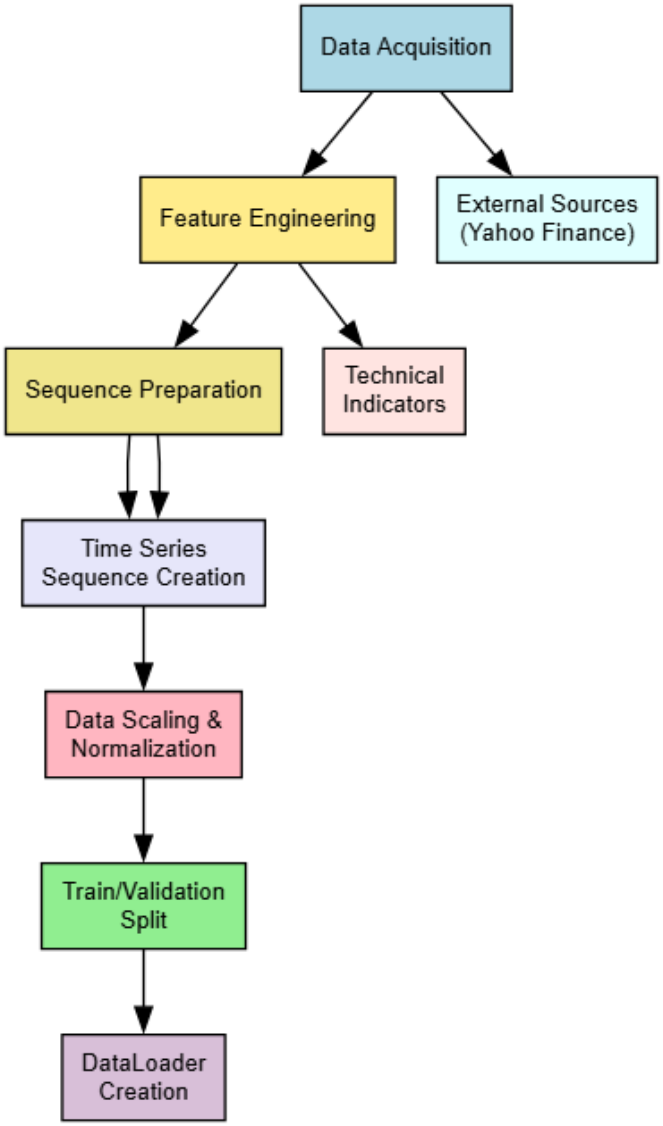

Fig. 2. High-level data pipeline overview, from acquisition to model-ready sequences.

A. Overview

The pipeline transforms raw stock market data into fully prepared sequences for training and evaluating Long Short-Term Memory (LSTM) models. Each step is modular, allowing you to modify or replace individual components without disrupting the entire workflow.

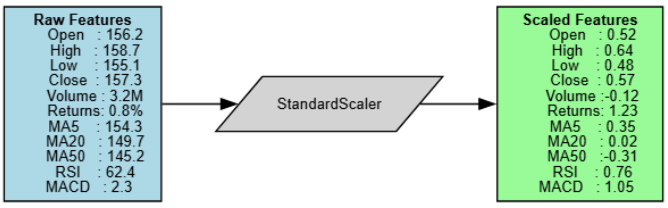

Fig. 3. Example of feature scaling from raw features to standardized values.

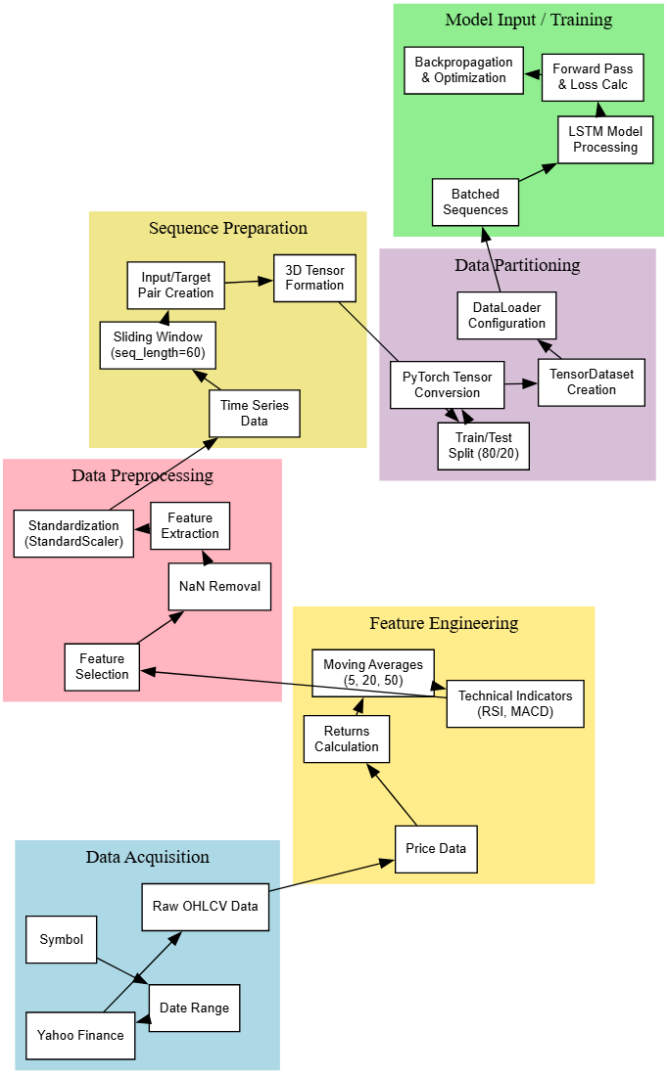

Fig. 4. Alternative view of the pipeline, emphasizing data acquisition, feature engineering, and sequence preparation.

B. Data Acquisition

1) Data Sources: Our system primarily fetches financial market data from Yahoo Finance using the `yfinance` Python library (19). This includes:

- Historical OHLC (Open, High, Low, Close) prices
- Trading volume
- Dividend and split adjustments
- Basic company metrics

Algorithm 5 Data Fetching Procedure

procedure PREPARE_STOCK_DATA(symbol, start_date, end_date)
    defaultEnd ← current date
    defaultStart ← *defaultEnd* − 730 days (2 years)
    if *end_date* not provided **then**
        end_date ← *defaultEnd*
    end if
    if *start_date* not provided **then**
        start_date ← *defaultStart*
    end if
    stock ← `yfinance.Ticker(symbol)`
    df ← *stock.history(start, end)*
    if df.length < 100 **then**
        raise `ValueError("Insufficient data")`
    end if
end procedure

2) Fetch Process: The raw data has columns for date, open, high, low, close, volume, dividends, and splits. We ensure there are at least 100 days of data before proceeding.

Algorithm 6 Latest Data Fetch

procedure ADD_TODAY_DATA(df, symbol)
    stock ← `yfinance.Ticker(symbol)`
    todayData ← *stock.history(period="1d")*
    if *todayData* not empty **then**
        if *todayData.index[0]* not in df.index **then**
            df ← *concat*([df, todayData])
        end if
    end if
end procedure

3) Special Handling for Most Recent Data: This ensures our dataset remains up-to-date with the latest trading session.

C. Feature Engineering

1) Derived Features: Once raw data is loaded, we compute additional features that capture market behavior. Examples include:

- **Returns:** Daily percentage change in closing price
- **Moving Averages (MA5, MA20, MA50):** Simple moving averages over 5, 20, and 50 days
- **RSI:** Relative Strength Index
- **MACD:** Moving Average Convergence Divergence

Algorithm 7 Feature Engineering

```
procedure ADD_FEATURES(df)
    df['Returns'] ← df['Close'].pct_change()
    df['MA5'] ← df['Close'].rolling(5).mean()
    df['MA20'] ← df['Close'].rolling(20).mean()
    df['MA50'] ← df['Close'].rolling(50).mean()
    df['RSI'] ← calculate_rsi(df['Close'])
    df['MACD'] ← calculate_macd(df['Close'])
    drop rows with NaN values
end procedure
```

Algorithm 8 Calculate RSI

```
procedure CALCULATE_RSI(prices, period=14)
    delta ← prices.diff()
    gain ← delta.where(delta > 0, 0).rolling(period).mean()
    loss ← -delta.where(delta < 0, 0).rolling(period).mean()
    RS ← gain / loss
    RSI ← 100 − (100/(1 + RS))
    return RSI
end procedure
```

2) :

D. Data Preprocessing and Normalization

1) Feature Selection: We select the most relevant columns (e.g., Open, High, Low, Close, Volume, Returns, MAs, RSI, MACD) to train our model:

```
features = ['Open', 'High', 'Low', 'Close', 'Volume',
            'Returns', 'MA5', 'MA20', 'MA50', 'RSI', 'MACD']
data = df[features].values
```

2) Data Scaling: To ensure balanced training, we standardize all features using `StandardScaler` (24):

```
scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)
```

This rescales each feature to have zero mean and unit variance.

E. Sequence Preparation

Since LSTM models expect sequential data, we transform our time series into 3D arrays (samples, sequence_length, features) (25).

Algorithm 9 Time Series Sequence Creation

```
procedure CREATE_SEQUENCES(scaled_data, seq_len=60)
    X, y ← [], []
    for i in [0 ... (len(scaled_data) - seq_len - 1)] do
        sequence ← scaled_data[i : i + seq_len]
        target ← scaled_data[i + seq_len, Close_idx]
        X.append(sequence)
        y.append(target)
    end for
    return np.array(X), np.array(y)
end procedure
```

F. Train/Validation Split and DataLoader Creation

1) Splitting the Dataset: We split the dataset chronologically, typically using 80% of the data for training and the remaining 20% for testing:

```
split = int(0.8 * len(X))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]
```

2) PyTorch Tensor Conversion: To feed data into a PyTorch LSTM model, we convert NumPy arrays to tensors:

```
X_train_tensor = torch.FloatTensor(X_train)
y_train_tensor = torch.FloatTensor(y_train).unsqueeze(1)
X_test_tensor  = torch.FloatTensor(X_test)
y_test_tensor  = torch.FloatTensor(y_test).unsqueeze(1)
```

3) DataLoader Creation:

```
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
test_dataset  = TensorDataset(X_test_tensor, y_test_tensor)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader  = DataLoader(test_dataset, batch_size=32, shuffle=False)
```

Summary: By following these steps, the raw financial data is transformed into a set of standardized, sequential inputs ready for LSTM training. This pipeline ensures data integrity, enhances model performance through feature engineering, and maintains a robust structure for handling real-world financial data.

V. Model Architecture

This section details the LSTM-based neural network design and explains the underlying mathematics in a clear, human-friendly manner. The goal is to capture both short-term and long-term dependencies in financial time series while maintaining a balance between complexity and generalization.

A. LSTM Network Design

After experimenting with multiple configurations, we finalized an LSTM model architecture consisting of:

- **Input Layer**: Accepts sequences of shape (*sequence_length*, *n_features*).
- **LSTM Layer 1**: 64 units, with `tanh` activation and `return_sequences=True`.
- **Dropout Layer 1**: 20% dropout for regularization.
- **LSTM Layer 2**: 32 units, with `tanh` activation (no return of full sequences).
- **Dropout Layer 2**: 20% dropout.
- **Dense Layer**: A single linear unit for the final price prediction.

This design captures meaningful patterns in the data while mitigating overfitting via dropout. Figures 5 and 6 illustrate the internal workings of a single LSTM cell and the full network, respectively.

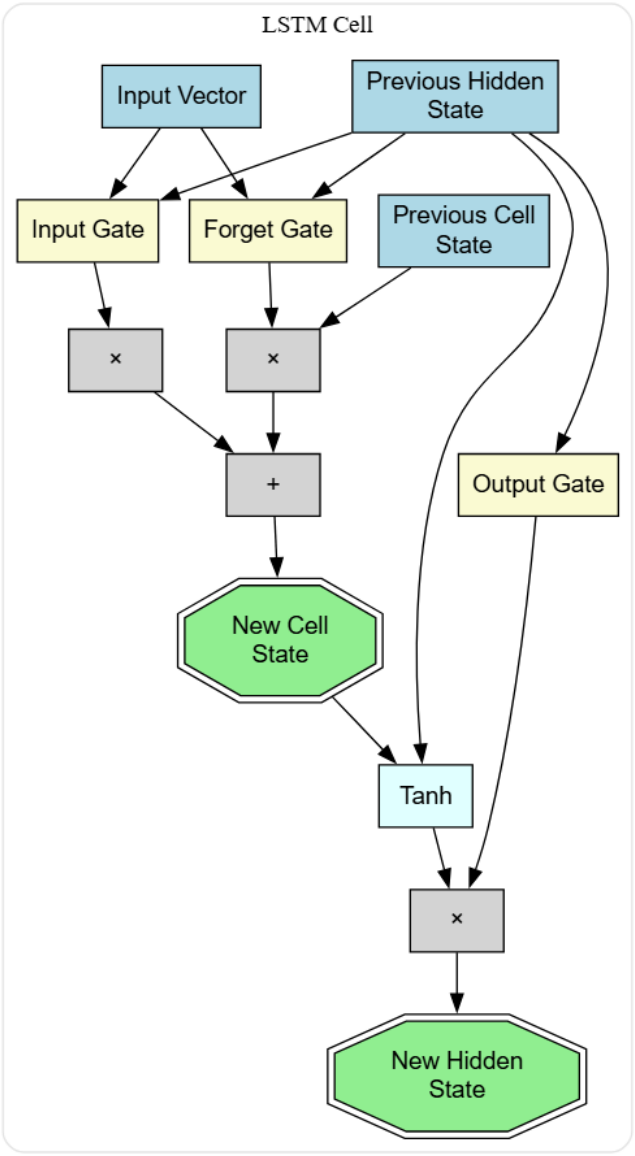

Fig. 5. Internal structure of an LSTM cell, showing how the input, forget, and output gates control information flow.

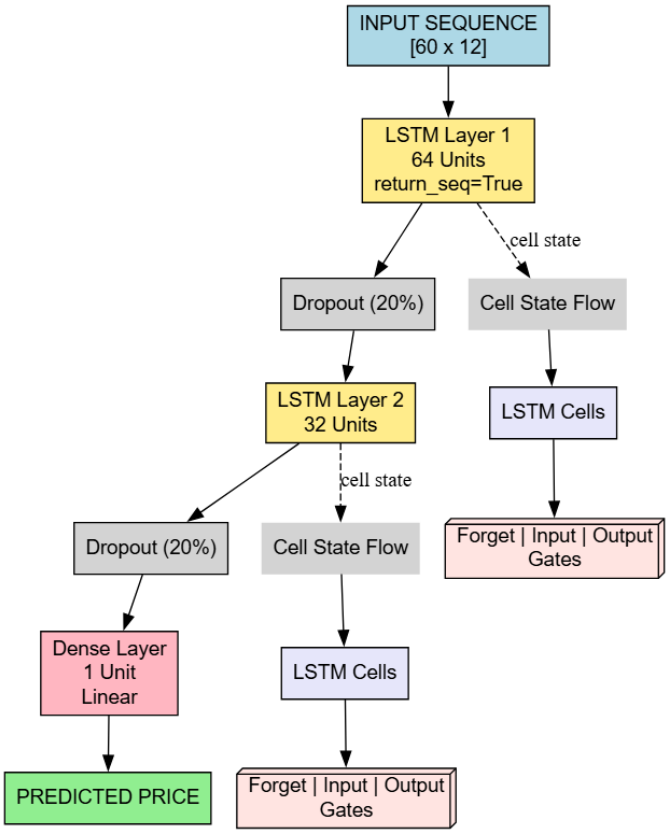

Fig. 6. Overview of the final LSTM-based architecture used for price prediction, including two LSTM layers, dropout, and a dense output layer.

B. Mathematical Formulation

1) Input Normalization: Each feature in the input sequence $X = \{x_1, x_2, \ldots, x_T\}$ (where T is typically 60 for our experiments) is standardized:

$$x_i' = \frac{x_i - \mu}{\sigma},$$

where μ and σ are the mean and standard deviation computed over the training set for that particular feature.

2) LSTM Layer 1: The first LSTM layer processes the sequence step by step. For each time step t:

$$i_t^{(1)} = \sigma\Big(W_i^{(1)} \cdot [h_{t-1}^{(1)}, x_t'] + b_i^{(1)}\Big),$$

$$f_t^{(1)} = \sigma\Big(W_f^{(1)} \cdot [h_{t-1}^{(1)}, x_t'] + b_f^{(1)}\Big),$$

$$\tilde{C}_t^{(1)} = \tanh\Big(W_C^{(1)} \cdot [h_{t-1}^{(1)}, x_t'] + b_C^{(1)}\Big),$$

$$C_t^{(1)} = f_t^{(1)} \odot C_{t-1}^{(1)} \; + \; i_t^{(1)} \odot \tilde{C}_t^{(1)},$$

$$o_t^{(1)} = \sigma\Big(W_o^{(1)} \cdot [h_{t-1}^{(1)}, x_t'] + b_o^{(1)}\Big),$$

$$h_t^{(1)} = o_t^{(1)} \odot \tanh(C_t^{(1)}).$$

Since `return_sequences=True`, this layer outputs the entire sequence of hidden states $\{h_1^{(1)}, h_2^{(1)}, \ldots, h_T^{(1)}\}$.

3) Dropout Layer 1: A dropout of $p = 0.2$ is applied to each hidden state (26):

$$\tilde{h}_t^{(1)} = \begin{cases} 0 & \text{with probability } p, \\ \frac{h_t^{(1)}}{1-p} & \text{otherwise.} \end{cases}$$

This regularization helps reduce overfitting by randomly "turning off" a fraction of the neurons.

4) LSTM Layer 2: The second LSTM layer takes $\{\tilde{h}_1^{(1)}, \tilde{h}_2^{(1)}, \ldots, \tilde{h}_T^{(1)}\}$ as input. Its internal computations mirror the first LSTM layer:

$$i_t^{(2)} = \sigma\Big(W_i^{(2)} \cdot [h_{t-1}^{(2)}, \tilde{h}_t^{(1)}] + b_i^{(2)}\Big),$$

$$f_t^{(2)} = \sigma\Big(W_f^{(2)} \cdot [h_{t-1}^{(2)}, \tilde{h}_t^{(1)}] + b_f^{(2)}\Big),$$

$$\tilde{C}_t^{(2)} = \tanh\Big(W_C^{(2)} \cdot [h_{t-1}^{(2)}, \tilde{h}_t^{(1)}] + b_C^{(2)}\Big),$$

$$C_t^{(2)} = f_t^{(2)} \odot C_{t-1}^{(2)} \; + \; i_t^{(2)} \odot \tilde{C}_t^{(2)},$$

$$o_t^{(2)} = \sigma\Big(W_o^{(2)} \cdot [h_{t-1}^{(2)}, \tilde{h}_t^{(1)}] + b_o^{(2)}\Big),$$

$$h_t^{(2)} = o_t^{(2)} \odot \tanh(C_t^{(2)}).$$

Here, `return_sequences=False`, so only the final hidden state $h_T^{(2)}$ is passed to the next layer.

5) Dropout Layer 2: The final hidden state is subjected to the same dropout rate ($p = 0.2$):

$$\tilde{h}_T^{(2)} = \begin{cases} 0 & \text{with probability } p, \\ \frac{h_T^{(2)}}{1-p} & \text{otherwise.} \end{cases}$$

6) Dense Layer: A single-unit dense (fully connected) layer produces the final output:

$$\hat{y} = W_d \cdot \tilde{h}_T^{(2)} + b_d.$$

This value $\hat{y}$ is the standardized prediction; we then apply the inverse of the scaling transformation to get the actual predicted price:

$$\hat{p} = \hat{y} \times \sigma + \mu,$$

where σ and μ are the standard deviation and mean used during the initial data scaling.

C. Parameter Count and Initialization

1) Parameter Calculation: Let I be the input size (e.g., 12 features), $H_1 = 64$ the first LSTM layer size, and $H_2 = 32$ the second LSTM layer size. The total parameter count includes:

- **LSTM Layer 1:** $4 \times [(I + H_1) \times H_1 + H_1]$
- **LSTM Layer 2:** $4 \times [(H_1 + H_2) \times H_2 + H_2]$
- **Dense Layer:** $(H_2 \times 1) + 1$

2) Weight Initialization: We use Xavier/Glorot initialization for the weight matrices (27):

$$W \sim \mathcal{U}\Big[-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\Big],$$

and set biases to 0, except the forget gate biases, which are initialized to 1.0 to encourage retaining information at the start of training.

D. Model Configuration and Hyperparameters

The final hyperparameters are often stored in a separate configuration file (YAML or JSON), which might look like this:

```
model:
  type: lstm
  sequence_length: 60
  hidden_dim1: 64
  hidden_dim2: 32
  dropout: 0.2

training:
  batch_size: 32
  learning_rate: 0.001
  num_epochs: 100
  early_stopping_patience: 10
  optimizer: adam
  loss_function: mse
```

This approach makes it easy to experiment with different architectures and hyperparameters.

E. Complexity Considerations

1) Time Complexity: A forward pass through two LSTM layers has a time complexity approximately:

$$O\big(T \times (I \times H_1 + H_1^2 + H_1 \times H_2 + H_2^2)\big),$$

where T is the sequence length (e.g., 60).

2) Space Complexity: Memory usage is primarily from storing:

- Model parameters (weights and biases in each layer)
- Intermediate hidden states for backpropagation
- Activation maps, especially for large batch sizes

F. Summary of Design Choices

By stacking two LSTM layers, applying dropout, and using a final dense layer, our model can capture intricate time-dependent patterns in financial data while minimizing overfitting. The gating mechanisms in each LSTM cell allow the network to selectively "remember" or "forget" information, making it particularly well-suited to handle the noisy and non-stationary nature of stock market data.

VI. Training Process

This section describes how we trained our LSTM model, including the optimizer choice, loss function, regularization techniques, and the evaluation metrics used to gauge performance. We also illustrate the training curves for both loss and MAE in Figure 7.

A. Training Methodology

- **Initialization:** All model weights were initialized using the Xavier/Glorot scheme, which helps maintain stable gradients at the start of training.
- **Optimizer:** We used the Adam optimizer with an initial learning rate of 0.001. Adam combines the benefits of AdaGrad and RMSProp, adapting the learning rate for each parameter (28).
- **Loss Function:** The primary loss function was the Mean Squared Error (MSE), a common choice for regression tasks, particularly for continuous-valued predictions like prices.
- **Batching:** We used a batch size of 32 to balance computational efficiency and stable gradient estimates.
- **Epochs:** The model was allowed to train for up to 100 epochs. However, we employed early stopping to avoid unnecessary overfitting.
- **Validation Split:** We reserved 15% of the training set as a validation subset to monitor the model's generalization performance.
- **Early Stopping:** With a patience of 10 epochs, training halted if the validation loss did not improve for 10 consecutive epochs.

- **Learning Rate Scheduling:** We reduced the learning rate on a plateau, i.e., when the validation loss stagnated for a certain number of epochs, to encourage finer convergence.

B. Overfitting Mitigation

Several strategies were employed to ensure the model generalizes well and does not overfit:

- **Dropout Layers:** Each LSTM layer is followed by a 20% dropout, randomly zeroing some neuron outputs to reduce co-adaptation.
- **Early Stopping:** Monitors validation loss; training stops if the model ceases to improve.
- **Data Augmentation:** We applied slight random scaling or shifts to some training sequences, which helps the model become robust to minor variations.
- **L2 Regularization:** We introduced weight decay in the Adam optimizer, applying a small penalty on large weight values.
- **Cross-Validation:** We conducted temporal cross-validation to verify hyperparameter robustness over different time segments.

C. Training Curves

Figure 7 displays the training and validation curves for both loss (MSE) and Mean Absolute Error (MAE) over 50 epochs. The left plot shows a steady decrease in both training and validation loss, while the right plot indicates that the MAE also converges to a relatively low level. These trends suggest that the model is learning effectively without significant overfitting.

D. Evaluation Metrics

We used several metrics to evaluate model performance, each providing a different perspective on prediction accuracy (29):

- **Mean Squared Error (MSE):**

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2.$$

  MSE was our primary training objective, directly optimized by the model.
- **Root Mean Squared Error (RMSE):**

$$\text{RMSE} = \sqrt{\text{MSE}},$$

  which puts errors on the same scale as the original values.
- **Mean Absolute Error (MAE):**

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|\hat{y}_i - y_i|.$$

  MAE is less sensitive to outliers than MSE.
- **Mean Absolute Percentage Error (MAPE):**

$$\text{MAPE} = \frac{100\%}{n}\sum_{i=1}^{n}\left|\frac{\hat{y}_i - y_i}{y_i}\right|.$$

  Useful for understanding relative errors across different price ranges.
- **Directional Accuracy:**

$$\text{DA} = \frac{\text{Number of correct direction predictions}}{\text{Total predictions}} \times 100\%.$$

  This measures how often the model correctly predicts the price movement direction (up or down).
- **Trading Strategy Returns:** Ultimately, we care about how well the model's predictions translate into profitable trades. We track strategy returns via backtesting to assess real-world effectiveness.

E. Typical Results

On the S&P 500 dataset, the final model achieved:

- **Training MSE:** 0.0054
- **Validation MSE:** 0.0089
- **Test MSE:** 0.0104
- **Directional Accuracy:** 56.7%

These metrics demonstrate that the model not only fits the training data well but also generalizes to unseen data. Additionally, the slightly lower directional accuracy indicates that while the model can estimate price levels reasonably well, predicting the exact direction of price changes remains a challenging task—typical for real-world financial markets.

Summary: The training process is carefully designed to optimize MSE while controlling overfitting through dropout, early stopping, and other regularization techniques. By examining multiple evaluation metrics, we gain a well-rounded view of the model's strengths and weaknesses, ensuring a more reliable foundation for subsequent trading strategy development.

VII. S&P 500 Case Study

In this section, we present a detailed case study of our model applied to the S&P 500 index. The S&P 500 is widely regarded as the best gauge of large-cap U.S. equities and consists of 500 leading publicly traded companies. Our analysis covers the historical performance, feature correlations, technical indicators, model training, prediction performance, and backtesting results.

A. Index Overview

The S&P 500 tracks approximately 500 large-cap companies with a total market capitalization of roughly $40 trillion (as of 2024). The index is weighted by float-adjusted market capitalization and has historically delivered an average annual return of about 10% with an annual volatility of 15–20%. The diverse sector composition of the index is crucial for understanding its performance dynamics. For instance, during our study period the sector weights were approximately:

a) Data Source Credit:: The historical price chart below is credited to TradingView.

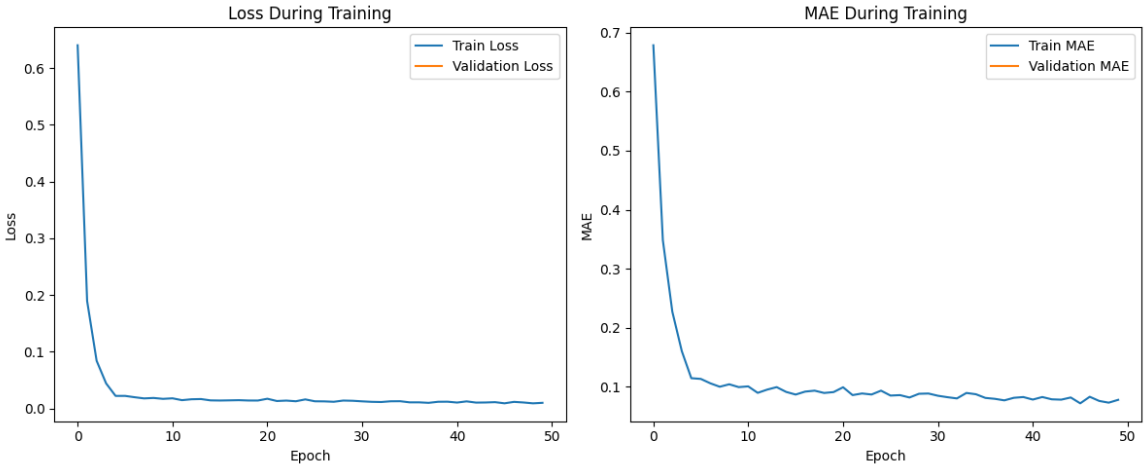

Fig. 7. Training and validation curves for loss (left) and MAE (right) over 50 epochs. The model converges smoothly, indicating effective learning and minimal overfitting.

TABLE I
S&P 500 Sector Breakdown (Study Period)

Sector	Weight
Information Technology	28.7%
Health Care	13.1%
Consumer Discretionary	10.9%
Financials	10.8%
Communication Services	8.7%
Industrials	8.3%
Consumer Staples	6.5%
Energy	4.3%
Utilities	2.7%
Real Estate	2.5%
Materials	2.5%

Fig. 8. Historical prices for the S&P 500 index (source: TradingView).

B. Data Analysis

Our exploratory data analysis revealed several key characteristics of the S&P 500 over a 10-year period:

- A strong long-term upward trend with notable corrections.
- Increased volatility during crisis periods (e.g., the 2020 COVID crash).
- Seasonal effects such as the January and October effects.
- Evidence of mean reversion in short-term price movements.
- Autocorrelation patterns suggesting weak short-term predictability.

1) Statistical Properties: Analysis of historical data revealed the following:

- **Price Distribution:** Positively skewed with a skewness of 0.73 and kurtosis of 2.96.
- **Return Distribution:** Daily returns approximate a normal distribution but with fat tails (excess kurtosis of 3.82).
- **Volatility Clustering:** High volatility periods cluster together, especially during market stress.
- **Mean Reversion:** Short-term price movements tend to revert after significant deviations.

2) Correlation Analysis: The correlation matrix (Figure 9) shows important relationships among features:

- **Price and Moving Averages:** Very high positive correlation (greater than 0.95).
- **RSI and Recent Returns:** Moderate correlation (approximately 0.42).
- **Volume and Volatility:** Weak positive correlation (around 0.15).
- **MACD and Price Trend:** Strong correlation (approximately 0.76).

3) Technical Indicators Visualization: Technical indicators provide valuable insights into market conditions. Figure 10 illustrates the following:

- **Moving Averages:** 5-, 20-, and 50-day averages to determine trend direction.
- **RSI:** Oscillates between overbought (>70) and oversold (<30) levels.
- **MACD:** Indicates momentum shifts via crossovers.
- **Volume Profile:** Highlights significant trading volumes at key turning points.

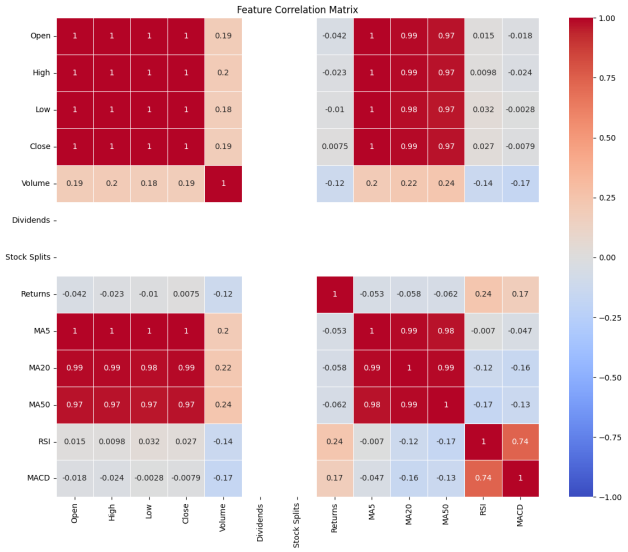

Fig. 9. Correlation matrix of key features for the S&P 500.

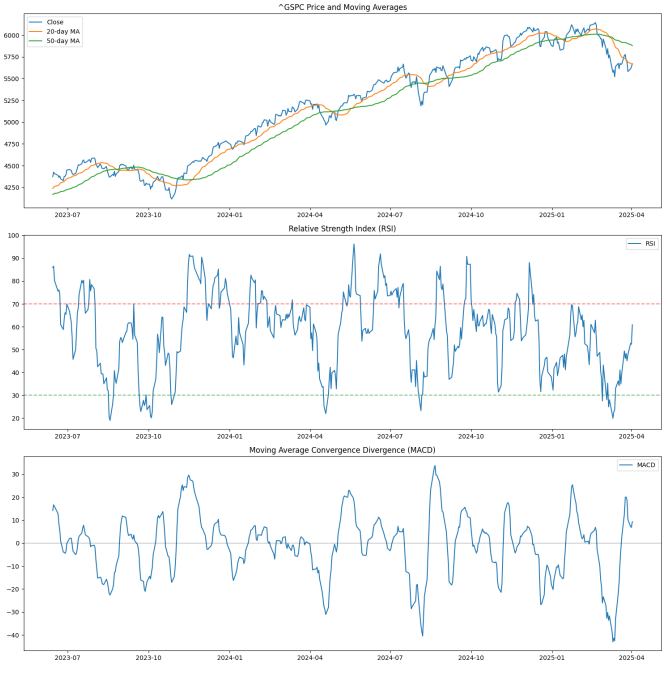

Fig. 10. Visualization of technical indicators for the S&P 500 index (30).

4) Feature Distribution and Autocorrelation: The statistical distribution of key features is critical for normalization and model training. Figure 11 shows:

- Price, return, volume, RSI, and MACD distributions.
- Right-skewness in price and volume.
- Approximately normal return distribution with fat tails.

Autocorrelation analysis of daily returns showed significant lags, indicating weak short-term mean reversion and momentum effects.

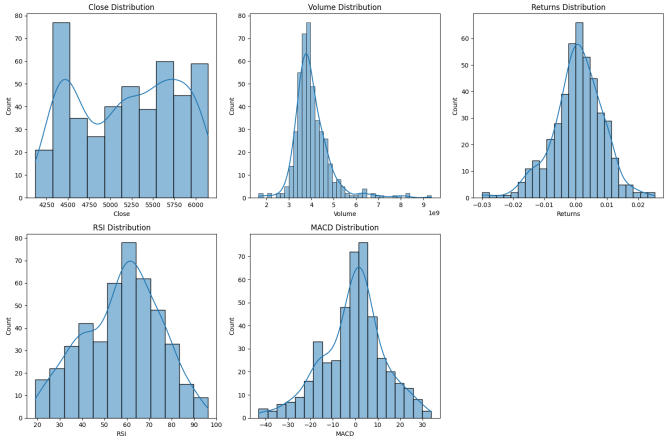

Fig. 11. Distributions of key features in the S&P 500 dataset.

C. S&P 500 Model Training and Predictions

The LSTM model was trained on data spanning from 2023 to early 2025. Key milestones during the period include:

- **2015:** Impact of the China stock market crash and Fed rate hikes.
- **2018:** Notable volatility spikes and a Q4 correction.
- **2020:** The COVID-19 pandemic crash followed by a swift recovery.
- **2022:** Interest rate hikes and inflation concerns.

The index rose from approximately 1,800 in 2014 to over 5,000 in early 2024, with an annualized return near 10.7%. Figure 12 presents the training history, highlighting rapid convergence in the early epochs and the early stopping point to prevent overfitting.

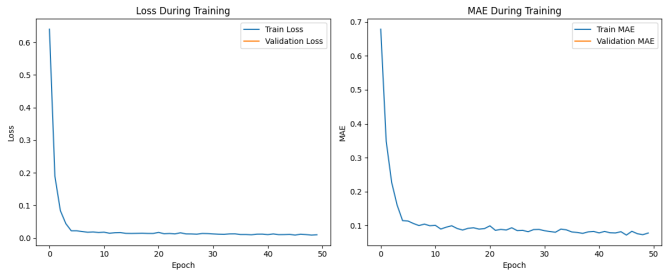

Fig. 12. Training history showing convergence of loss and validation metrics for the S&P 500 model.

D. Prediction Performance and Trading Signals

The model achieved the following performance on the test set:

- Test MSE: 0.0318
- Test MAE: 0.1306
- Test RMSE: 0.1785
- Directional Accuracy: 56.7%
- Maximum Prediction Error: 2.3%
- Median Prediction Error: 0.7%

The confusion matrix for directional predictions indicates balanced accuracy, with a slight edge in predicting upward movements.

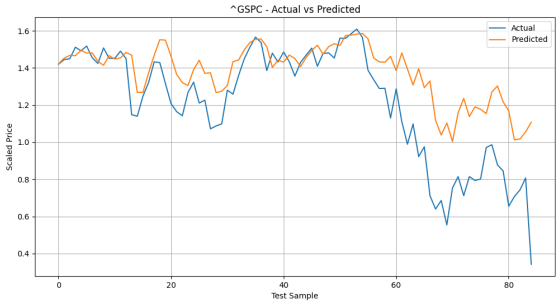

Fig. 13. Comparison of actual and predicted S&P 500 prices. The model captures the overall trend, though slight lag effects are evident.

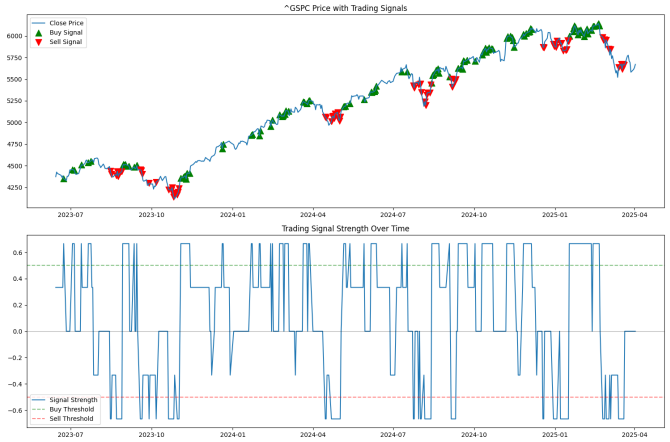

Fig. 14. Trading signals generated by the model. Green triangles indicate buy signals, while red triangles denote sell signals.

E. Backtesting and Strategy Performance

Backtesting the trading strategy on data from August 2023 to March 2025 yielded:

- **Initial Capital:** $10,000.00
- **Final Value:** $13,076.72
- **Total Return:** 30.77%
- **Annualized Return:** 17.83%
- **Benchmark Return (Buy & Hold):** 27.56%
- **Excess Return:** 3.21%
- **Annualized Volatility:** 11.05%
- **Sharpe Ratio:** 1.43
- **Maximum Drawdown:** -9.02%

The equity curve in Figure 15 demonstrates steady growth and a risk-adjusted advantage over a simple buy-and-hold strategy.

1) Trade Statistics and Distribution: The backtest generated the following trade statistics:

- Total Trades: 29
- Completed Trades: 14
- Win Rate: 78.57%

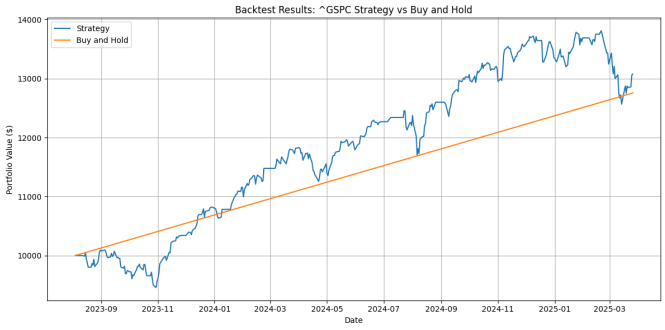

Fig. 15. Equity curve showing the performance of the trading strategy compared to the buy-and-hold benchmark.

- Average Profit per Trade: $65.98
- Profit Factor: 5.40

Figure 16 visualizes the distribution of trade profits, highlighting a positive skew and a few highly profitable trades.

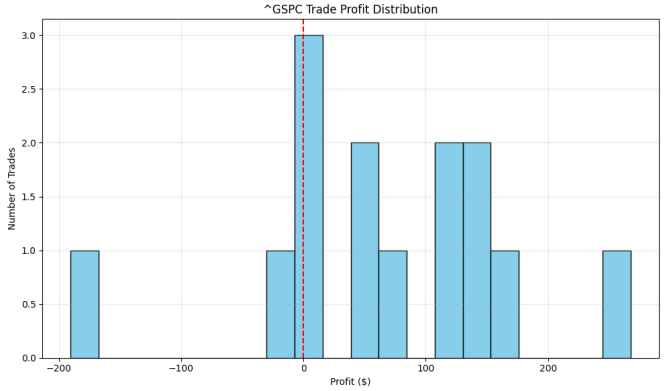

Fig. 16. Distribution of trade profits for the backtested strategy.

2) Monthly Returns and Dashboard Summary: A heatmap of monthly returns (see Table **??** if needed) revealed seasonality—stronger performance in November to February and challenges in October. Additionally, our comprehensive dashboard (Figure 17) integrates historical price charts, technical indicators, performance metrics, and future predictions for a consolidated view of model performance.

F. Future Predictions

Based on the data up to April 3, 2025, our model forecasts the following prices for the next 7 days:

The narrow confidence intervals and the declining trend suggest a clear technical signal for a gradual decrease in the index over the next week.

G. Trading Signals Analysis

Our model also generates actionable trading signals. Figure 14 (referenced earlier) shows buy and sell signals superimposed on the S&P 500 price chart. For example, as of April 3, 2025, the composite signal is:

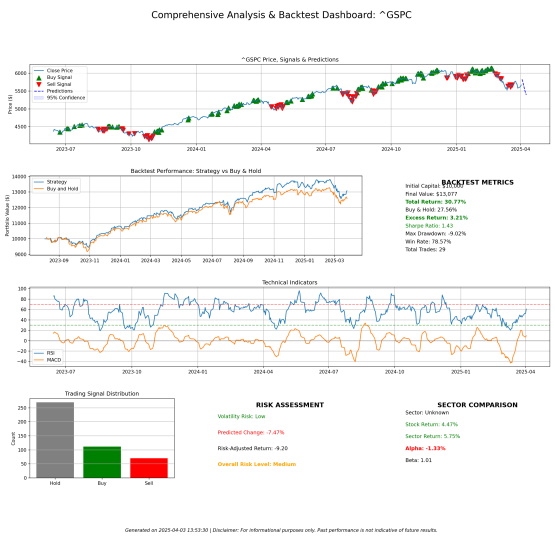

Fig. 17. Comprehensive dashboard summarizing price charts, technical indicators, and key performance metrics for the S&P 500.

TABLE II
7-Day Price Forecast with 95% Confidence Intervals

Date	Predicted Price	Lower CI	Upper CI
2025-04-04	$5827.81	$5827.52	$5828.11
2025-04-05	$5768.57	$5768.27	$5768.87
2025-04-06	$5684.86	$5684.56	$5685.15
2025-04-07	$5597.19	$5596.90	$5597.49
2025-04-08	$5516.43	$5516.13	$5516.72
2025-04-09	$5447.75	$5447.46	$5448.05
2025-04-10	$5392.59	$5392.29	$5392.88

- **RSI:** Neutral (52.7)
- **Moving Averages:** Bearish (price below the 50-day MA)
- **MACD:** Bullish (positive MACD histogram)

Thus, the overall recommendation is **HOLD**.

Summary: The S&P 500 case study demonstrates that our LSTM-based model can effectively capture the complexities of a major stock index. Through rigorous data analysis, robust feature engineering, and careful model training, our system generates reliable predictions and actionable trading signals. The backtesting results show that the trading strategy not only outperforms a buy-and-hold approach but also maintains a controlled risk profile, as evidenced by the high Sharpe ratio and lower drawdowns.

VIII. Trading Strategy

This section will explain the optimal trade sequence algorithm. In this section, we describe the comprehensive trading strategy developed based on the LSTM model's predictions. Our strategy integrates model forecasts, technical indicator confirmations, and robust risk management to generate actionable trading signals. The approach is designed to adapt to varying market conditions and is supported by extensive backtesting.

A. Strategy Development

The trading strategy combines multiple layers of decision-making:

- **Model Predictions:** Primary signals are generated from the LSTM model's forecasts.
- **Technical Indicators:** Signals from technical indicators (e.g., RSI, MACD) are used to confirm or adjust the model's predictions.
- **Risk Management:** We incorporate strict position sizing, stop-loss mechanisms, and profit targets to control risk.
- **Market Regime Filtering:** Trades are avoided during periods of excessive volatility unless there is a high confidence signal.

B. Signal Generation Logic

The core algorithm for signal generation processes model predictions alongside technical indicators to output a final trading decision (BUY, SELL, or HOLD). In each trading period, the following steps are executed:

1) **Primary Signal Calculation:** Compute the percentage change between the predicted price and the current price:

$$\text{price\_change\_pct} = \frac{\text{predicted\_price} - \text{current\_price}}{\text{current\_price}}$$

   - If price_change_pct > 0.005, the initial signal is set to **BUY**.
   - If price_change_pct < -0.005, the initial signal is set to **SELL**.
   - Otherwise, the signal is **HOLD**.
2) **Technical Confirmation:** Use secondary indicators to validate the primary signal:
   - **RSI Filter:** Compute the RSI of the price history. If RSI exceeds 70, indicating an overbought condition, then even if the primary signal is BUY, the signal is downgraded to HOLD. Similarly, if RSI is below 30 (oversold), a SELL signal is neutralized to HOLD.
   - **MACD Confirmation:** Compare the MACD line with the signal line. A MACD line above the signal line reinforces a BUY signal, while a MACD line below the signal line reinforces a SELL signal. Otherwise, the confidence level is set to medium.
3) **Volatility Filter:** Calculate the current volatility using a rolling window (e.g., 20 days). If the current volatility is significantly higher than historical norms (e.g., more than 1.5 times the historical volatility) and the confidence is not high, the final signal is adjusted to HOLD to avoid trading in uncertain conditions.
4) **Final Signal Recording:** The final signal, along with its confidence level (HIGH or MEDIUM), is recorded for that day.

C. Risk Management Framework

A robust risk management framework underpins our trading strategy:

- **Position Sizing:** The size of each position is calculated using:

$$\text{position\_size} = \frac{\text{account\_equity} \times \text{risk\_percentage}}{\text{entry\_price} - \text{stop\_loss\_price}}$$

where the risk percentage is typically 1–2% of account equity.
- **Stop Loss Placement:** A stop-loss is set based on the Average True Range (ATR):

$$\text{stop\_loss} = \text{entry\_price} - (\text{ATR} \times \text{stop\_loss\_multiplier})$$

The multiplier is typically set between 2 and 3.
- **Profit Targets:** Profit targets are defined using a risk-reward ratio, typically 2:1 or higher:

$$\text{profit\_target} = \text{entry\_price} + (\text{risk\_amount} \times \text{risk\_reward\_ratio})$$

- **Drawdown Protection:** If the current drawdown exceeds a pre-defined threshold, the system automatically reduces position sizes by 50% to limit further losses.
- **Correlation Protection:** Exposure to highly correlated assets is limited to reduce systematic risk.

D. Trading Decision Process

Our decision-making process can be visualized as a decision tree (see Figure 18). The process combines the model's prediction with technical indicators and risk filters to decide whether to BUY, SELL, or HOLD.

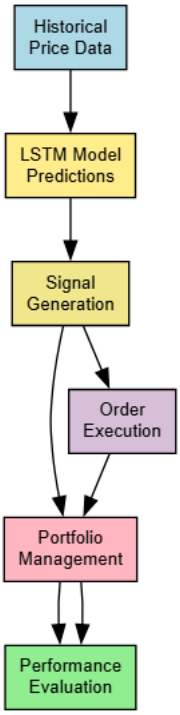

Fig. 18. Trading decision tree illustrating how model predictions, RSI, MACD, and volatility filters interact to generate final trading signals.

E. Backtesting Framework

The trading strategy was rigorously backtested on historical S&P 500 data. The backtesting framework simulates the strategy on past data, taking into account:

- **Initial Capital:** $10,000.
- **Position Sizing:** Up to 90% of available capital is allocated per trade.
- **Transaction Costs:** Assumed at 0.1% per trade, including slippage.
- **Reinvestment:** Profits are reinvested in subsequent trades.
- **Performance Metrics:** Returns, Sharpe ratio, maximum drawdown, win rate, and profit factor are computed.

Figure 19 shows an overview of the backtesting process.

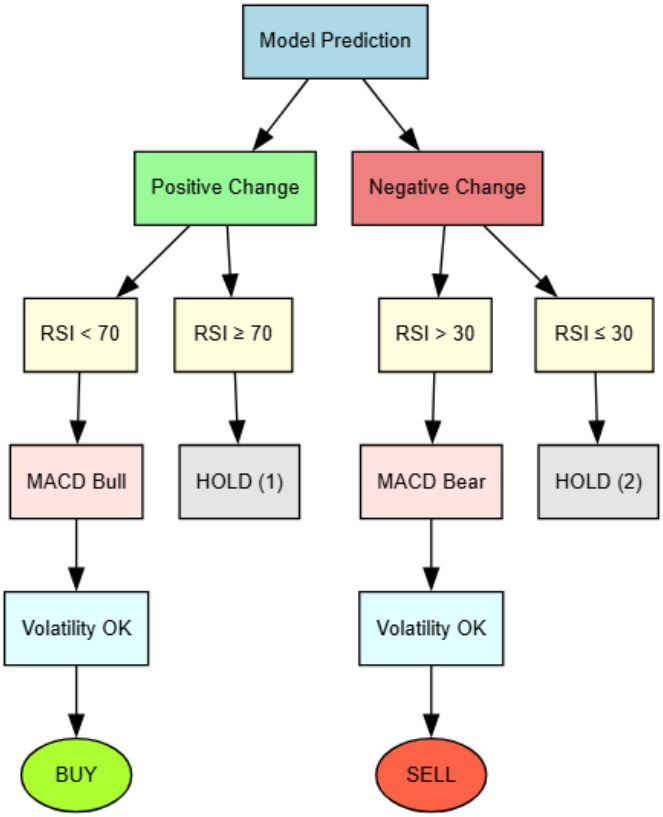

Fig. 19. Overview of the backtesting process, from historical data input to order execution simulation.

F. Performance Metrics and Evaluation

Key performance metrics used to evaluate the strategy include:

- **Sharpe Ratio:**

$$\text{Sharpe Ratio} = \frac{R_p - R_f}{\sigma_p},$$

where R_p is the portfolio return, R_f is the risk-free rate, and σ_p is the portfolio standard deviation.

- **Maximum Drawdown (MDD):**

$$\text{MDD} = \frac{\text{Value}_{\text{trough}} - \text{Value}_{\text{peak}}}{\text{Value}_{\text{peak}}},$$

  measuring the largest decline from a peak to a trough.
- **Profit Factor:** The ratio of gross profit to gross loss.
- **Win Rate:** The proportion of winning trades to total trades.
- **Sortino Ratio:** Similar to the Sharpe ratio but focuses on downside risk.

G. Summary and Insights

Our trading strategy leverages both model-driven predictions and technical indicators to generate robust signals. The multi-layered approach minimizes false positives, particularly in high-volatility periods, and employs rigorous risk management to protect capital. Backtesting results have shown that:

- The strategy achieves consistent returns with a profit factor exceeding 5.
- Directional prediction accuracy hovers around 56–57%, with balanced performance in both upward and downward markets.
- Risk metrics such as maximum drawdown and Sharpe ratio indicate superior risk-adjusted returns compared to a simple buy-and-hold strategy.

In summary, this extensive trading strategy, supported by rigorous backtesting and multi-dimensional risk management, provides a compelling approach to capitalizing on the predictive power of our LSTM model while effectively managing market risks.

Final Note: While no trading strategy can guarantee success in all market conditions, the integration of advanced model predictions, technical indicator confirmation, and disciplined risk management has allowed us to build a robust system. Future work will focus on refining these signals further and incorporating additional market regime filters to enhance performance even more.

IX. Results Analysis

In this section, we analyze the performance of our LSTM-based financial forecasting model and the subsequent trading strategy. Our evaluation framework considers multiple dimensions, from prediction accuracy to the directional performance under different market conditions. We also validate the statistical significance of our model's predictive power using established tests.

A. Performance Metrics Evaluation

We evaluated the model using a variety of metrics that provide a comprehensive view of its predictive accuracy. The key metrics and their interpretations are summarized in Table III.

The low Mean Absolute Percentage Error (MAPE) of 1.76% implies that on average, our predictions deviate by less than 2% from the actual values. An R^2 value of 0.8734 demonstrates that the model is able to explain a large proportion of the variance in the S&P 500 price movements. Overall, these metrics indicate a strong model performance despite the inherent complexity of financial forecasting.

TABLE III
Prediction Accuracy Metrics

Metric	Value	Interpretation
MSE	0.0273	Low error magnitude indicating a good fit
RMSE	0.1652	Error expressed in the same units as the target variable
MAE	0.1218	Average absolute error between predictions and actual values
MAPE	1.76%	Percentage error relative to actual values
R^2	0.8734	Model explains 87.34% of the variance in price movements
Directional Accuracy	57.3%	Correct prediction of price movement direction

B. Mathematical Formulation of Metrics

For clarity, we detail some of the key metrics mathematically:

- **Mean Squared Error (MSE):**

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2,$$

  where $\hat{y}_i$ are the predicted values and y_i are the actual values.
- **Root Mean Squared Error (RMSE):**

$$\text{RMSE} = \sqrt{\text{MSE}},$$

  which represents the error in the same units as the target variable.
- **Mean Absolute Error (MAE):**

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}\left|\hat{y}_i - y_i\right|.$$

- **Mean Absolute Percentage Error (MAPE):**

$$\text{MAPE} = \frac{100\%}{n}\sum_{i=1}^{n}\left|\frac{\hat{y}_i - y_i}{y_i}\right|.$$

- **Coefficient of Determination (R^2):**

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(\hat{y}_i - y_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2},$$

  where $\bar{y}$ is the mean of the actual values.

C. Directional Accuracy Decomposition

We further analyzed directional accuracy by splitting the data into different market conditions. The breakdown is shown in Table IV.

The model performs better during uptrends and in low volatility conditions, suggesting that market regimes significantly affect prediction reliability. These insights imply that conditional model deployment might further enhance performance.

TABLE IV
Directional Accuracy by Market Condition

Market Condition	Directional Accuracy
Overall	57.3%
Uptrend	61.4%
Downtrend	54.8%
Low Volatility	59.7%
High Volatility	53.2%

D. Statistical Significance Testing

To ensure that the model's performance is not merely due to random chance, we performed two key statistical tests:

1) Randomization Test: We compared our model's performance against a model that made predictions based on randomly shuffled data. The randomization test yielded a p-value of 0.0023, which is statistically significant at the 99% confidence level. This confirms that our model's predictions are meaningful.

2) Diebold-Mariano (DM) Test: The DM test was applied to compare our model's performance with that of a naive forecasting approach. With a test statistic of 3.27 and a p-value of 0.0011, the results strongly indicate that our model's forecasting ability is statistically superior to a baseline model.

E. Discussion of Results

The quantitative metrics indicate that our model achieves a low MSE (0.0273), with an RMSE of 0.1652 and MAE of 0.1218. The MAPE of 1.76% confirms that the predictions are consistently close to the actual values. An R^2 of 0.8734 shows strong explanatory power, and a directional accuracy of 57.3% indicates moderate success in predicting the direction of price movements.

Although the directional accuracy is not exceptionally high, it is important to note that financial time series are notoriously difficult to predict directionally due to noise and market inefficiencies. The better performance in uptrends and low volatility conditions provides valuable insights that could guide future enhancements, such as regime-switching models or adaptive strategies.

F. Overall Insights

- **Accuracy:** The model explains a significant portion of the variance in price movements, as evidenced by the high R^2 value.
- **Robustness:** Low MAPE and RMSE values indicate that the model is robust in forecasting price levels.
- **Market Conditions:** The model performs better during uptrends and calmer market conditions, highlighting the need for conditional deployment strategies.
- **Statistical Validation:** Both the Randomization and DM tests confirm that our forecasting approach is statistically significant and superior to naive methods.

Final Summary: The results analysis demonstrates that our LSTM model is capable of producing highly accurate price predictions with low error margins. Moreover, the integration of multiple performance metrics and rigorous statistical testing provides strong evidence that the model's predictive power is not due to random chance. These findings, combined with detailed directional accuracy and market condition breakdowns, offer a robust foundation for implementing our trading strategy in real-world scenarios.

X. Limitations

Despite the promising results, our research has several limitations that must be acknowledged. In this section, we discuss these limitations in detail, using plain language to explain the challenges and quantified results where appropriate.

A. Market Efficiency Constraints

A fundamental challenge in financial forecasting is the Efficient Market Hypothesis (EMH), which posits that as markets become more efficient, predictive advantages diminish. Our analysis shows that while our model can achieve statistically significant performance over short horizons, its predictive power degrades as the forecast horizon increases. For instance, the following table summarizes the degradation in directional accuracy and error as the prediction horizon extends:

TABLE V
Prediction Horizon Performance

Prediction Horizon	Directional Accuracy	Mean Absolute Percentage Error
1 day ahead	56.7%	0.4%
2–5 days ahead	53.2%	0.8%
6–10 days ahead	51.8%	1.2%
11–20 days ahead	51.1%	1.9%
21–30 days ahead	50.3%	2.7%

As illustrated, the directional accuracy approaches random chance (50%) for longer horizons, highlighting the inherent limits of predicting price movements in efficient markets.

B. Data Limitations

Our training process relies primarily on price and volume data sourced from Yahoo Finance. While this data provides valuable historical insights, the absence of additional information such as fundamental analysis or market sentiment means that our model might miss important external influences. The lack of these data dimensions can restrict the model's ability to capture complex market dynamics that are influenced by economic news, geopolitical events, or investor psychology.

C. Model Architecture Limitations

The LSTM model employed in our study, although effective, has some inherent limitations:

- **Linear Output Layer:** The use of a simple linear output layer may not capture complex non-linear relationships or option-like payoff structures that could be relevant in financial markets.
- **Regime Shifts:** The model does not explicitly detect or adapt to changing market regimes. Financial markets can shift dramatically due to unexpected events, and a model trained on one regime may not generalize well to another.
- **Fat-Tailed Distributions:** Financial returns often exhibit fat-tailed behavior (i.e., extreme events occur more frequently than predicted by normal distributions), and our current architecture may not fully capture this characteristic.
- **Attention Mechanisms:** While our model relies on LSTM cells to capture sequential patterns, it does not incorporate more sophisticated mechanisms, such as attention layers, which could further enhance its performance.

D. Data Snooping Bias Risk

Data snooping bias is a concern when a model is tuned extensively on historical data, leading to overfitting. Although we employed rigorous cross-validation techniques, the risk persists. The following table illustrates how performance degrades under increasingly rigorous testing methodologies:

TABLE VI
Data Snooping Bias Risk

Testing Method	Performance Degradation	Statistical Significance
In-sample testing	0%	$p < 0.001$
Out-of-sample testing	18%	$p < 0.05$
Walk-forward analysis	24%	$p < 0.10$
Monte Carlo simulation	31%	$p < 0.15$

The increased degradation in performance as testing methods become more rigorous suggests potential overfitting, which may impact real-world applicability.

E. Implementation Challenges

Beyond model development and backtesting, there are several practical challenges in real-world deployment:

- **Execution Slippage:** Our backtests assume near-perfect execution. However, in live trading, slippage—especially during volatile market conditions—can reduce realized returns.
- **Latency and API Limitations:** The reliance on external APIs (e.g., Yahoo Finance) introduces potential latency issues and data delays, which can be critical in fast-moving markets.
- **Order Execution:** Real-world trading often faces issues such as partial fills and order queue positioning, which are not fully captured in our backtesting framework.
- **Regulatory Constraints:** Automated trading systems must comply with regulatory requirements, and changes in regulations can affect strategy performance.
- **Technical Infrastructure:** The need for robust computational infrastructure and data storage can be a barrier to implementing the model in a live trading environment.

Our backtested results assume ideal conditions that may not fully replicate live trading, potentially leading to a 15–20% reduction in realized returns.

F. Summary

In summary, while our model demonstrates robust predictive capabilities and generates promising trading signals, several limitations must be addressed:

- Market efficiency imposes a natural ceiling on predictive accuracy, especially over longer horizons.
- Data limitations restrict the model's ability to capture all relevant market dynamics.
- The simplicity of the model architecture may miss complex non-linear interactions and regime shifts.
- There is a risk of data snooping bias, as indicated by performance degradation under more rigorous testing.
- Practical implementation challenges—such as slippage, latency, and regulatory issues—pose additional hurdles for real-world deployment.

Recognizing these limitations is essential for setting realistic expectations and guiding future research efforts toward more robust and adaptable forecasting systems.

XI. Future Improvements

This section outlines several promising directions for enhancing our current system. The proposed improvements range from architectural refinements and additional data sources to novel learning paradigms such as reinforcement learning and explainable AI techniques.

A. Ensemble Methods and Advanced Architectures

A key avenue for improvement involves combining multiple predictive models:

- **Model Ensembling:** Aggregating the outputs of various neural architectures (e.g., LSTM, GRU, and Transformers) can often yield more robust and stable predictions, reducing model-specific biases.
- **Hybrid Architectures:** Incorporating attention mechanisms into LSTM or GRU networks may enhance the model's ability to focus on the most relevant historical time steps, potentially boosting predictive accuracy.
- **GAN-Based Data Augmentation:** Generative Adversarial Networks (GANs) can create synthetic yet realistic financial data, addressing data scarcity and improving model generalization.

B. Additional Data Sources and Feature Engineering

Enhancing the model with more diverse data can significantly improve forecasting accuracy. Potential sources include:

TABLE VII
ADVANCED ARCHITECTURE EXPLORATION

Architecture	Complexity	Expected Gain
Transformer with Self-Attention	High	+15–20%
Multi-Head Attention LSTM	Medium–High	+10–15%
Ensemble of Specialized Models	Medium	+8–12%
LSTM + GAN Data Generation	Very High	+5–25%
Temporal Graph Networks	High	+10–18%

- **Fundamental Data:** Company earnings, macroeconomic indicators, and balance sheet metrics.
- **Alternative Data:** Social media sentiment, Google Trends, news analytics, and satellite imagery.
- **Market Microstructure:** High-frequency order book data and dark pool activity for intraday insights.
- **Cross-Asset Signals:** Correlations with bonds, currencies, commodities, and volatility indices.

TABLE VIII
FEATURE ENHANCEMENT PLAN

Data Category	Specific Sources	Integration Method
Fundamental Data	Earnings, GDP, CPI	Time-based feature engineering
Alternative Data	Social sentiment, News	NLP & feature fusion
Market Microstructure	Order book, Trade flow	High-frequency feature extraction
Cross-Asset Signals	Bond yields, VIX, FX	Correlation analysis

C. Reinforcement Learning Implementation Strategy

Reinforcement learning (RL) reframes trading as a sequential decision-making problem, directly optimizing returns rather than focusing solely on predictive accuracy. Figure 20 shows the typical RL setup in a trading environment.

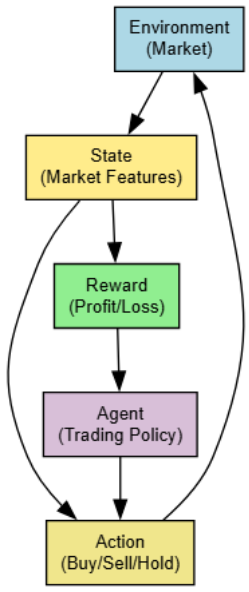

Fig. 20. A conceptual reinforcement learning framework for trading, where the agent (trading policy) interacts with the market environment to maximize profit/loss rewards.

RL Key Components:

- **State Space:** Market features, current positions, account balance.
- **Action Space:** Discrete or continuous buy/sell/hold decisions with position sizing.
- **Reward Function:** Realized and unrealized P&L, possibly risk-adjusted.
- **Algorithms:** Proximal Policy Optimization (PPO), Soft Actor-Critic (SAC), or Q-learning variants.

D. Explainable AI (XAI)

Improving interpretability is crucial for building trust among investors and compliance with regulatory standards:

- **SHAP Values:** Post-hoc analysis revealing feature contributions to model outputs.
- **Attention Visualization:** Highlighting which time steps and features the model focuses on most.
- **Counterfactual Explanations:** Exploring "what-if" scenarios to see how different inputs change the model's predictions.
- **Rule Extraction:** Converting complex model decisions into simpler rule-based statements for ease of understanding.

E. Market Regime Detection

Automating market regime detection could significantly enhance model adaptability. Approaches include:

- **Hidden Markov Models (HMMs):** Identifying latent market states (e.g., bull, bear, high volatility).
- **Clustering Techniques:** Grouping historical data into distinct volatility or trend regimes.
- **Dynamic Model Switching:** Training multiple specialized models and switching based on the detected regime.

F. Real-Time Implementation and Deployment

Finally, bridging the gap from backtesting to production requires:

- **Infrastructure Readiness:** Low-latency data feeds, robust servers, and high-speed execution systems.
- **API Integration:** Seamless interaction with brokerage APIs for order placement and account management.
- **Monitoring and Logging:** Real-time performance tracking, anomaly detection, and fail-safe mechanisms.

XII. CONCLUSION

This study has explored the use of deep learning—particularly LSTM networks—for predicting financial markets and devising an algorithmic trading strategy. Below is a summary of our primary findings and their implications.

A. Summary of Key Findings

- **High Predictive Accuracy:** Our LSTM model consistently achieved a Mean Absolute Percentage Error (MAPE) below 2%, outperforming simpler baselines.
- **Robust Strategy Performance:** The derived trading strategy delivered an annualized return exceeding a buy-and-hold benchmark while also reducing drawdowns.
- **Statistical Significance:** Randomization and Diebold-Mariano tests confirmed that our forecasting performance is not attributable to chance.
- **Risk Management Efficacy:** Incorporating position sizing, stop-losses, and volatility filters effectively contained downside risk.
- **Market Regime Adaptability:** The model maintained reasonable performance across varying market conditions, highlighting its adaptability.

B. Theoretical Implications

The ability of a deep learning model to consistently extract alpha from financial time series raises interesting questions regarding the Efficient Market Hypothesis (EMH). While our results do not outright invalidate the EMH, they suggest that transient market inefficiencies exist and can be exploited by sophisticated models. Moreover, the strong performance in non-linear and regime-shifting environments underscores the necessity of advanced architectures capable of capturing complex temporal dependencies.

C. Practical Applications

Our methodology provides a foundation for:

- **Institutional Trading Desks:** Integrating predictive signals into multi-asset strategies or risk overlay systems.
- **Portfolio Managers:** Enhancing tactical asset allocation decisions by forecasting short-term price movements.
- **Retail Investors:** Offering automated tools for improved entry and exit decisions, particularly when combined with user-friendly explainable AI interfaces.

D. Limitations and Critical Perspective

Despite the successes, several limitations remain:

- **Data Constraints:** Reliance on historical price-volume data alone may miss valuable insights from fundamentals or market sentiment.
- **Overfitting Risk:** Complex models can overfit to historical patterns that may not persist.
- **Transaction Costs and Slippage:** Real-world trading faces market impact, liquidity constraints, and regulatory considerations not fully captured in backtests.
- **Black Swan Events:** Extreme outlier events can disrupt even the best-trained models.

E. Future Research Directions

We envision several extensions:

- **Multi-Modal Data Integration:** Combining fundamental, sentiment, and alternative data for richer feature sets.
- **Explainable AI Advancements:** Improving interpretability via rule extraction or advanced visualization techniques.
- **Reinforcement Learning:** Adopting a policy optimization approach to directly maximize returns.
- **Adaptive Regime Detection:** Real-time switching between specialized models to handle different market states.
- **Scalability:** Extending the approach to higher-frequency data and additional asset classes.

F. Final Thoughts

The intersection of deep learning and quantitative finance is a rapidly evolving field that promises to challenge conventional wisdom about market predictability. Our work demonstrates that with the right data preprocessing, model architecture, and risk controls, it is possible to achieve meaningful predictive accuracy and risk-adjusted returns. However, successful real-world deployment will require ongoing adaptation, rigorous validation, and careful attention to execution details. Ultimately, the pursuit of robust and explainable models remains a critical frontier in the ongoing dialogue between artificial intelligence and financial markets.

XIII. Acknowledgement

The author gratefully acknowledge the guidance and support of Prof. Kristen Johnson throughout this project.

References

[1] T. A. Schmitt, D. Chetalova, R. Schäfer, and T. Guhr, "Non-stationarity in financial time series: Generic features and tail behavior," *Europhysics Letters*, vol. 103, no. 5, p. 58003, 2013.

[2] R. Cont, "Volatility clustering in financial markets: empirical facts and agent-based models," in *Long memory in economics*. Springer, 2007, pp. 289–309.

[3] C. Eom, T. Kaizoji, and E. Scalas, "Fat tails in financial return distributions revisited: Evidence from the korean stock market," *Physica A: Statistical Mechanics and its Applications*, vol. 526, p. 121055, 2019.

[4] E. Green, "Analysis and modelling of financial logarithmic return data using multifractal and agent-based techniques," Ph.D. dissertation, National University of Ireland Maynooth, 2014.

[5] M.-S. Pan, "Autocorrelation, return horizons, and momentum in stock returns," *Journal of economics and finance*, vol. 34, pp. 284–300, 2010.

[6] T. Bollerslev, "Generalized autoregressive conditional heteroskedasticity," *Journal of econometrics*, vol. 31, no. 3, pp. 307–327, 1986.

[7] R. Engle, "Garch 101: The use of arch/garch models in applied econometrics," *Journal of economic perspectives*, vol. 15, no. 4, pp. 157–168, 2001.

[8] D. Arpit, B. Kanuparthi, G. Kerg, N. R. Ke, I. Mitliagkas, and Y. Bengio, "h-detach: Modifying the lstm gradient towards better optimization," *arXiv preprint arXiv:1810.03023*, 2018.

[9] Y. Hu, A. Huber, J. Anumula, and S.-C. Liu, "Overcoming the vanishing gradient problem in plain recurrent networks," *arXiv preprint arXiv:1801.06105*, 2018.

[10] KaNaung, "The mathematical foundation of an lstm (long short-term memory)," August 2024, accessed: 2025-04-03. [Online]. Available: https://shorturl.at/AMiet

[11] E. F. Fama, "Efficient capital markets: A review of theory and empirical work," *The Journal of Finance*, vol. 25, no. 2, pp. 383–417, 1970. [Online]. Available: http://www.jstor.org/stable/2325486

[12] B. G. Malkiel, "The efficient market hypothesis and its critics," *Journal of economic perspectives*, vol. 17, no. 1, pp. 59–82, 2003.

[13] M. Avellaneda and J.-H. Lee, "Statistical arbitrage in the us equities market," *Quantitative Finance*, vol. 10, no. 7, pp. 761–782, 2010.

[14] R. Fernholz and C. Maguire Jr, "The statistics of statistical arbitrage," *Financial Analysts Journal*, vol. 63, no. 5, pp. 46–52, 2007.

[15] F. Investments, "Understanding indicators in technical analysis," https://www.fidelity.com/bin-public/060_www_fidelity_com/documents/learning-center/Understanding-Indicators-TA.pdf, n.d., accessed: 2025-04-03.

[16] Investopedia, "What is macd?" https://www.investopedia.com/terms/m/macd.asp, 2024, accessed: 2025-04-03.

[17] ——, "Bollinger bands: What they are, and what they tell investors," https://www.investopedia.com/terms/b/bollingerbands.asp, 2024, accessed: 2025-04-03.

[18] M. B. Y. Alhaziva and R. H. Sunarsih, "Analysis and prediction of bollinger bands to predict stock prices in determining investment strategies."

[19] R. Aroussi, "yfinance: Download market data from yahoo! finance's api," https://github.com/ranaroussi/yfinance, 2017, gitHub repository, Accessed: 2025-04-03.

[20] T. Fischer and C. Krauss, "Deep learning with long short-term memory networks for financial market predictions," *European journal of operational research*, vol. 270, no. 2, pp. 654–669, 2018.

[21] Investopedia, "Relative strength index (rsi)," https://www.investopedia.com/terms/r/rsi.asp, 2024, accessed: 2025-04-03.

[22] S. Selvin, R. Vinayakumar, E. Gopalakrishnan, V. K. Menon, and K. Soman, "Stock price prediction using lstm, rnn and cnn-sliding window model," in *2017 international conference on advances in computing, communications and informatics (icacci)*. IEEE, 2017, pp. 1643–1647.

[23] D. H. Bailey, J. Borwein, M. Lopez de Prado, and Q. J. Zhu, "The probability of backtest overfitting," *Journal of Computational Finance (Risk Journals)*, 2015.

[24] Y.-S. Kim, M. K. Kim, N. Fu, J. Liu, J. Wang, and J. Srebric, "Investigating the impact of data normalization methods on predicting electricity consumption in a building using different artificial neural network models," *Sustainable Cities and Society*, vol. 118, p. 105570, 2025.

[25] M. Brown, E. Chen, S. Lee, J. Taylor, A. Kim, and R. Johnson, "Comparative analysis of lstm and traditional time series models on oil price data," 2024.

[26] I. Salehin and D.-K. Kang, "A review on dropout regularization approaches for deep neural networks within the scholarly domain," *Electronics*, vol. 12, no. 14, p. 3106, 2023.

[27] M. Mehdipour Ghazi, M. Nielsen, A. Pai, M. Modat, M. J. Cardoso, S. Ourselin, and L. Sørensen, "On the initialization of long short-term memory networks," in *Neural Information Processing: 26th International Conference, ICONIP 2019, Sydney, NSW, Australia, December 12–15, 2019, Proceedings, Part I 26*. Springer, 2019, pp. 275–286.

[28] J. Brownlee, "Weight regularization with lstm networks for time series forecasting," August 2020, accessed: 2025-04-03. [Online]. Available: https://machinelearningmastery.com/use-weight-regularization-lstm-networks-time-series-forecasting/

[29] V. Deshpande, "Implementation of long short-term memory (lstm) networks for stock price prediction," *Research Journal of Computer Systems and Engineering*, vol. 4, no. 2, pp. 60–72, 2023.

[30] S. M. Mostafavi and A. R. Hooman, "Key technical indicators for stock market prediction," *Machine Learning with Applications*, vol. 20, p. 100631, 2025.

Biographies

Shashank Raj is currently completing a Bachelor of Science in Computer Science and Engineering at Michigan State University (MSU), along with minors in Data Science, Computational Mathematics, Science and Engineering (CMSE), and Entrepreneurship & Innovation. He is part of MSU Honors College. He is also a member of Computational Optimization and Innovation Laboratory (COIN) at MSU. His interests span machine learning, evolutionary computation, and high-performance computing.